

JOGO DE CORRIDA

Tutorial Completo do Código

Corrida Velocidade Total



HTML + CSS + JavaScript | Pista pseudo-3D | 8 pilotos | 4 cenários | 3 carros

Objetivo técnico

Este material explica como o Jogo de Corrida foi construído, por que cada sistema existe e onde localizar os principais trechos no código. A organização segue os três arquivos centrais: index.html, style.css e game.js, conectando interface, aparência, pista pseudo-3D, física, inteligência artificial, colisões, áudio, HUD e persistência.

Os números de linha citados correspondem aos arquivos comentados presentes na pasta do jogo. Como o código possui comentários didáticos extensos, as funções aparecem em intervalos grandes; as referências servem para chegar diretamente ao bloco responsável por cada sistema.

Mapa do tutorial

#	Capítulo	O que você encontra
1	Arquitetura	Responsabilidade de HTML, CSS, JavaScript e assets.
2	HTML	Menu, tela de corrida, HUD, controles e modais.
3	CSS	Tema, layout, HUD, responsividade, modais e controles móveis.
4	JavaScript base	DOM, constantes, configurações, estado, jogador e rivais.
5	Dados locais e áudio	IndexedDB e Web Audio API.
6	Pista pseudo-3D	Segmentos, curvas, elevação e projeção em perspectiva.
7	Renderização	Cenários, pista, carros, câmeras e efeitos.
8	Física do participante	Aceleração, freio, turbo, drift, pista e velocidade.
9	IA competitiva	Antecipação de curvas, turbo, recuperação e ultrapassagem.
10	Colisões	Deteção, contato lateral, traseiro e engavetamentos.
11	Corrida e ranking	Voltas, posição, resultado, pontos e IndexedDB.
12	Loop e eventos	Passo fixo, requestAnimationFrame, teclado, toque e botões.
13	Onde modificar	Mapa prático para alterar carros, pistas, dificuldade e interface.
14	Testes	Checklist para validar alterações sem quebrar os sistemas integrados.

1. Arquitetura geral

O jogo foi dividido em camadas para separar estrutura, apresentação e comportamento. Essa divisão reduz acoplamento visual e facilita localizar mudanças: o HTML declara os elementos, o CSS define como eles aparecem e o JavaScript controla tudo que muda durante a execução.

Camada	Arquivo	Responsabilidade
Estrutura	index.html	Menu, formulários, Canvas principal, HUD, minimapa, controles de toque, janelas e botões.
Apresentação	style.css	Tema, tipografia, layout, posicionamento, responsividade, efeitos do HUD, modais e interface móvel.
Lógica	game.js	Pista, perspectiva, renderização, física, IA, colisões, áudio, ranking, estados e ciclo de atualização.
Imagem	assets/capa_corrida.png	Composição visual usada como capa do menu.

Fluxo de execução

1. O navegador carrega index.html e encontra a interface completa.
2. O HTML carrega style.css para aplicar o tema e o posicionamento.
3. No final do HTML, game.js é carregado.
4. O JavaScript localiza elementos do DOM pelos IDs e prepara constantes, dados e eventos.
5. Ao clicar em INICIAR CORRIDA, as escolhas do menu são lidas e a pista é criada.
6. O loop principal atualiza física, IA, colisões e voltas em passo fixo.
7. A renderização desenha cenário, pista, carros, HUD e minimapa em cada quadro visível.
8. Ao terminar, o resultado é calculado e gravado no IndexedDB.

2. index.html: estrutura da interface

O HTML funciona como esqueleto do jogo. Ele não calcula física nem IA; sua função é disponibilizar os elementos que o JavaScript manipula e os componentes que o CSS estiliza.

Bloco	Localização	Por que existe
Menu principal	index.html, a partir da linha 77	Reúne título, descrição, campos de configuração e botões.
Nome do piloto	linha 153	Fornece o identificador que aparece no HUD e no ranking.
Modo de jogo	linha 188	Permite escolher controle manual ou piloto automático.
Dificuldade	linha 236	Seleciona o conjunto de parâmetros da IA.
Voltas	linha 292	Define o número de comprimentos de pista necessários para terminar.
Cenário	linha 359	Seleciona a configuração de pista e o desenho de fundo.
Carro	linha 423	Escolhe estatísticas, cor e tipo visual do veículo.
Botão iniciar	linha 568	Dispara iniciarCorrida().
Tela da corrida	linha 623	Contém Canvas e sobreposições do jogo.
HUD	linha 644	Agrupa posição, volta, placar, turbo, tempo, velocidade e câmera.
Minimapa	linha 836	Canvas secundário usado para desenhar a distribuição dos pilotos.
Como jogar	linha 1009	Modal com comandos principais.
Ranking	linha 1134	Modal preenchido dinamicamente com resultados agregados.
Pausa	linha 1207	Modal para continuar, reiniciar ou sair.
Resultado	linha 1276	Modal que recebe os dados finais da corrida.
Carga do JavaScript	linha 1408	Inclui game.js depois que os elementos HTML já existem.

Por que usar IDs

IDs como hudPosicao, carro, cenario e btnJogar criam pontos de ligação estáveis entre HTML e JavaScript. O código pode localizar exatamente o elemento que deve receber texto, responder a um clique ou fornecer uma configuração selecionada.

Exemplo de configuração declarada no HTML

```
<select id="dificuldade" class="campo">
  <option value="facil">Fácil</option>
  <option value="medio" selected>Médio</option>
  <option value="avancado">Avançado</option>
</select>
```

O valor selecionado não contém a lógica da dificuldade. Ele apenas fornece uma chave, como medio, que será utilizada no objeto CONFIG_DIFICULDADE dentro de game.js.

3. style.css: aparência e responsividade

O CSS transforma a estrutura simples do HTML em uma interface de corrida. O arquivo começa com variáveis de cor em :root, define tela cheia para o aplicativo e depois organiza menu, campos, botões, HUD, velocímetro, turbo, modais e adaptações para telas menores.

Seletor / região	Localização	Função
:root	linha 5	Centraliza cores reutilizáveis como fundo, painel, borda, amarelo, azul e vermelho.
.tela / .tela.ativa	linhas 130-139	Alterna visibilidade entre menu e jogo.
.tela-menu	linha 145	Define o fundo e a apresentação do menu.
.campo	linha 342	Padroniza inputs e selects.
.preview-selecao	linha 392	Organiza a prévia do carro e do cenário.
.botao	linha 582	Estiliza botões e estados de interação.
.hud	linha 693	Cria a camada fixa por cima do Canvas.
.painel	linha 735	Define o visual angular dos painéis.
.placar-pilotos	linha 802	Formata a lista de posições.
.turbo-barra	linha 913	Cria a barra de carga do turbo.
.velocimetro	linha 1029	Formata a leitura de velocidade.
.modal	linha 1401	Base para janelas de pausa, ranking, ajuda e resultado.
@media	a partir da linha 1723	Adapta menu, HUD e controles a diferentes tamanhos e dispositivos.

Por que a barra de turbo usa transform

A largura visual da carga é atualizada por transform: scaleX(). Essa técnica evita alterar a largura real do elemento em cada quadro e reduz recálculos de layout. O JavaScript apenas muda a escala horizontal, mantendo o restante do HUD estável.

Trecho conceitual do indicador de turbo

```
.turbo-barra i {
  width: 100%;
  transform: scaleX(1);
  transform-origin: left center;
  will-change: transform;
}
```

4. game.js: referências ao DOM e constantes

O JavaScript começa localizando o Canvas principal, o minimapa e todos os elementos que serão atualizados. Fazer essas referências uma vez evita repetir `document.getElementById` durante cada quadro.

Canvas principal e Canvas do minimapa

```
const canvas = document.getElementById('gameCanvas');
const ctx = canvas.getContext('2d');
const miniMapa = document.getElementById('miniMapa');
const miniCtx = miniMapa.getContext('2d');
```

Constantes de mundo e simulação

Constante	Valor	Papel
SEGMENTO	200	Comprimento de cada trecho usado para construir a pista.
LARGURA_PISTA	2000	Largura lógica da estrada no mundo pseudo-3D.
FOV	95	Campo de visão usado no cálculo de perspectiva.
DISTANCIA_DESENHO	280	Quantidade aproximada de segmentos à frente considerada na renderização.
PASSO_FIXO	1/60	Intervalo de simulação usado no loop de física.
LARGURA_COLISAO_CARRO	0.30	Limite lateral da caixa lógica de colisão.
COMPRIMENTO_COLISAO_CARRO	330	Limite longitudinal de proximidade para contato.

Essas constantes foram separadas porque representam parâmetros globais do mundo. Ajustá-las pode afetar muitos sistemas ao mesmo tempo, portanto são mantidas em um ponto central.

5. Objetos de configuração: dificuldade, carros, cenários e câmeras

CONFIG_DIFICULDADE

Cada chave concentra os fatores que a função `atualizarIA()` precisa para modificar o comportamento dos adversários. Isso evita espalhar condicionais de fácil, médio e avançado por toda a lógica.

Parâmetro	Efeito
<code>velocidadeIA</code>	Multiplica a velocidade-alvo dos rivais.
<code>errolA</code>	Controla pequenas variações e imprecisões.
<code>agressividade</code>	Influencia ultrapassagem, ritmo e intensidade de decisões.
<code>aceleracaoIA</code>	Multiplica a capacidade de aceleração.
<code>turboIA</code>	Aumenta probabilidade e intensidade de uso do turbo.
<code>recuperacao</code>	Controla ganho progressivo de ritmo de rivais distantes.
<code>premio</code>	Base usada no cálculo de pontos ao terminar.

CARROS

O objeto `CARROS` funciona como um catálogo. Cada entrada armazena nome, cor, tipo visual, velocidade máxima, velocidade com turbo, aceleração, frenagem, controle e texto de estatísticas. A física do participante e a IA consultam esse mesmo catálogo, mantendo os valores consistentes.

CENARIOS

O objeto `CENARIOS` guarda nome, descrição, tipo e paleta da pista. `construirPista()` utiliza `cenarioAtual` para selecionar o traçado; `desenharCenario()` usa a mesma chave para escolher o fundo correspondente.

CAMERAS

O vetor `CAMERAS` reúne altura, recuo, escala e posição visual do carro. A terceira câmera também define `carro:false`, o que faz `renderizar()` desenhar um capô no lugar do carro externo.

6. Estado global da corrida

Variáveis como estado, totalVoltas, dificuldadeAtual, cenárioAtual, carroAtual e modoAtual descrevem a corrida ativa. O estado pode assumir menu, contagem, corrida, pausado ou fim. Essa máquina de estados simples impede que a física seja atualizada em momentos inadequados.

Estado	O que acontece
menu	A tela inicial está ativa; nenhuma corrida é atualizada.
contagem	A contagem regressiva está visível; o loop desenha, mas ainda não move os carros como corrida ativa.
corrida	Física, IA, colisões e voltas são atualizadas.
pausado	O cronômetro de pausa é acumulado e a simulação não avança.
fim	O participante terminou e a tela de resultado é exibida.

Objeto jogador

O objeto jogador guarda tudo que muda durante a condução: nome, posição lateral x, distância percorrida, velocidade, turbo, drift, velocidade máxima, tempo final, pontos, temporizadores de colisão e variáveis utilizadas pelo piloto automático.

Vetor adversarios

Os sete rivais são criados em criarAdversarios(), linhas 5207-5380. Cada um recebe nome, cor, carro, posição de largada, velocidade, personalidade, intervalo de mudança de faixa, reserva de turbo, impulso inicial e tendência de uso do turbo.

7. Banco local com IndexedDB

O objeto Banco encapsula toda a persistência. Ele abre um banco chamado CorridaVelocidadeTotalDB e cria o object store resultados com chave numérica automática. Também define índices por nome e data.

Ideia central da criação do banco local

```
const req = indexedDB.open('CorridaVelocidadeTotalDB', 1);

if (!db.objectStoreNames.contains('resultados')) {
  const store = db.createObjectStore('resultados', {
    keyPath: 'id',
    autoIncrement: true
  });
}
```

Métodos

Método	Responsabilidade
abrir()	Abre o banco e cria a estrutura quando necessário.
salvar(resultado)	Adiciona uma corrida concluída em transação de escrita.
listar()	Recupera todos os resultados para montar o ranking.
limpar()	Remove todos os registros do object store.

O uso de Promises e async/await permite aguardar operações assíncronas do IndexedDB sem bloquear a interface.

8. Sistema de áudio com Web Audio API

AudioJogo cria som sem depender de arquivos externos. Um oscilador contínuo simula o motor e pequenos osciladores temporários produzem bipes de colisão, turbo, volta e vitória.

Motor

`atualizarMotor(normalizado)` muda frequência do oscilador, frequência do filtro e volume conforme a velocidade. O valor normalizado varia aproximadamente de 0 a 1, fazendo o som subir de tom à medida que o carro acelera.

Efeitos

Função	Uso
<code>bip()</code>	Base genérica para criar um oscilador temporário.
<code>colisao()</code>	Som grave e curto durante contatos.
<code>turbo()</code>	Som agudo e breve durante o uso do turbo.
<code>volta()</code>	Sequência de tons quando uma nova volta começa.
<code>vitoria()</code>	Sequência ascendente de quatro notas para primeiro lugar.

9. Funções matemáticas auxiliares

Antes de construir a pista, o código define funções pequenas que são reutilizadas em vários sistemas. `limitar()` mantém números dentro de um intervalo; `interpolar()` calcula valores intermediários; `easeIn`, `easeOut` e `easeInOut` criam transições suaves; `mod()` produz módulo positivo; `formatarTempo()` converte milissegundos para o formato de corrida.

Utilitários reutilizados em física, desenho e minimapa

```
const limitar = (v, min, max) => Math.max(min, Math.min(max, v));  
const interpolar = (a, b, t) => a + (b - a) * t;  
const mod = (n, m) => ((n % m) + m) % m;
```

Essas funções existem para reduzir repetição e garantir que diferentes partes do jogo usem as mesmas regras numéricas.

10. Construção da pista pseudo-3D

A pista é formada por uma sequência de segmentos. Cada segmento possui dois pontos no mundo, um valor de curva e referências de câmera e tela. Essa técnica permite criar uma sensação de profundidade sem usar uma biblioteca 3D.

Função	Linhas	Responsabilidade
adicionarSegmento()	2017-2122	Cria um segmento com pontos inicial e final, curva e cor alternada.
adicionarTrecho()	2123-2200	Combina entrada, trecho constante e saída, suavizando curva e elevação.
construirPista()	2201-2579	Monta sequências diferentes para cada cenário e calcula comprimento total.
acharSegmento()	2580-2612	Converte uma distância z no segmento correspondente.

Conceito usado para evitar mudanças bruscas de traçado

```
function adicionarTrecho(entrada, manter, saida, curva=0, elevacao=0) {
  // entrada: curva cresce suavemente
  // manter: curva permanece estável
  // saida: curva volta suavemente para zero
}
```

Por que dividir em entrada, manter e saída

Se uma curva começasse com o valor máximo de um segmento para o outro, a pista pareceria quebrada. As funções de easing fazem a curvatura aumentar e diminuir progressivamente, criando transições visuais e físicas mais naturais.

11. Projeção em perspectiva

A função `projetar()`, linhas 2613-2690, transforma coordenadas do mundo em coordenadas de tela. Primeiro calcula a posição do ponto em relação à câmera; depois aplica a profundidade de câmera derivada do campo de visão; finalmente obtém *x*, *y* e largura projetada no Canvas.

Essa etapa é a base do pseudo-3D: objetos distantes recebem escala menor e aparecem mais próximos do horizonte; objetos próximos recebem escala maior.

Desenho dos segmentos

`desenharSegmento()`, linhas 2760-2978, recebe pontos já projetados e desenha faixas de grama, asfalto, zebras e marcações. Os segmentos são processados do fundo para a frente para respeitar a sobreposição visual.

12. Cenários desenhados no Canvas

Os fundos não são uma única imagem fixa. O JavaScript contém funções específicas para cada ambiente, combinando gradientes, sol, montanhas, vegetação, prédios, luzes ou rochas.

Função	Linhas	Ambiente
<code>desenharCenarioCosta()</code>	3277-3509	Costa, pôr do sol, montanhas e elementos marítimos.
<code>desenharCenarioSerra()</code>	3510-3650	Serra, floresta e montanhas.
<code>desenharCenarioNeon()</code>	3651-3943	Cidade noturna, edifícios e luzes.
<code>desenharCenarioCanion()</code>	3944-4145	Deserto, cânions e formações rochosas.
<code>desenharCenario()</code>	4146-4197	Seleciona a função correta conforme <code>cenarioAtual</code> .

Separar uma função por cenário torna cada ambiente independente. É possível enriquecer um cenário sem alterar os demais ou incluir um novo tipo adicionando seus dados, sua pista e sua função de desenho.

13. Desenho dos carros

`desenharCarro()`, linhas 4682-5131, monta os veículos com formas geométricas no Canvas. O parâmetro `tipo` permite variações visuais entre `super`, `gt` e `sport`; `cor` define a identidade; `jogadorLocal` permite aplicar detalhes específicos ao carro do participante; `inclinacao` transmite esterço visual.

A largura do carro é calculada conforme a escala de perspectiva. Por isso um rival distante fica pequeno e aumenta gradualmente ao se aproximar da câmera.

Câmera de capô

`desenharCapo()`, linhas 5132-5206, é usada quando a configuração de câmera indica `carro:false`. Em vez de desenhar o veículo inteiro atrás da câmera, o código cria uma forma frontal na parte inferior da tela.

14. Criação da grade e início da corrida

criarAdversarios() - linhas 5207-5380

A função cria os sete rivais com distâncias iniciais e posições laterais alternadas. A grade foi espaçada para que as caixas de colisão não comecem sobrepostas. Além disso, cada rival recebe pequenas diferenças de personalidade e decisão.

iniciarCorrida() - linhas 5381-5596

Essa função é o ponto de preparação da prova. Ela lê campos do menu, escolhe configurações válidas, chama construirPista(), reinicializa o objeto jogador, cria rivais, zera cronômetros, troca as telas, ajusta Canvas e inicia a contagem.

Leitura e validação das opções do menu

```
totalVoltas = Number(voltasEl.value) || 3;
dificuldadeAtual = dificuldadeEl.value in CONFIG_DIFICULDADE ? dificuldadeEl.value : 'medio';
cenarioAtual = cenarioEl.value in CENARIOS ? cenarioEl.value : 'costa';
carroAtual = carroEl.value in CARROS ? carroEl.value : 'veloce';
```

iniciarContagem() - linhas 5597-5731

Mostra 3, 2, 1 e JÁ! usando setInterval. O estado continua em contagem até o encerramento, quando passa para corrida e registra os instantes usados pelo cronômetro e pela primeira volta.

15. Piloto automático do carro selecionado

`obterControleAutomatico()`, linhas 5732-6028, não move o carro diretamente. Em vez disso, retorna o mesmo tipo de comandos que um jogador humano produziria: acelerar, frear, esquerda, direita, turbo e drift. `atualizarJogador()` consome esse objeto sem precisar duplicar a física.

Decisões principais

- Analisa curvas aproximadamente 900 e 2200 unidades à frente.
- Calcula uma curva prevista combinando os dois segmentos.
- Procura um adversário à frente ocupando faixa semelhante.
- Escolhe um alvo lateral para contornar o bloqueio.
- Reduz quando a velocidade excede a velocidade segura da curva.
- Ativa turbo em trechos pouco curvos quando há carga suficiente.
- Pode usar drift em curvas intensas quando precisa alterar rapidamente a posição lateral.

Esse desenho foi feito para reaproveitar toda a física do participante e manter comportamento coerente entre modo manual e modo automático.

16. Física do participante

atualizarJogador(), linhas 6029-6454, é o núcleo da condução. A função recebe dt, consulta o carro selecionado e decide se os comandos virão do teclado/toque ou do piloto automático.

Etapa	O que o código faz
Entrada	Lê acelerar, frear, esquerda, direita, turbo e drift.
Limites	Converte max e turboMax do carro para unidades internas.
Aceleração	Soma carro.aceleracao * dt quando acelerando.
Resistência	Reduz velocidade quando o acelerador não está pressionado.
Frenagem	Subtrai carro.frenagem * dt.
Turbo	Soma aceleração extra e reduz a carga; fora do turbo, recupera carga.
Direção	Move jogador.x conforme velocidade, drift e controle do carro.
Curva da pista	Aplica deslocamento lateral proporcional à curva e à velocidade.
Fora da pista	Aplica penalidades de velocidade ao ultrapassar limites laterais.
Drift	Aumenta direção e aplica uma perda adicional de velocidade.
Distância	Avança jogador.distancia pelo produto velocidade * dt.
Áudio	Atualiza o motor proporcionalmente à velocidade.

Princípio da integração simples usada na movimentação

```
if (controle.acelerar) jogador.velocidade += carro.aceleracao * dt;
if (controle.frear) jogador.velocidade -= carro.frenagem * dt;

jogador.distancia += jogador.velocidade * dt;
```

Por que multiplicar por dt

dt representa o tempo transcorrido na simulação. Multiplicar aceleração, frenagem e deslocamento por dt evita que a física dependa diretamente da quantidade de quadros por segundo.

17. Colisões físicas

O sistema de colisão está dividido em funções menores para separar verificação, resolução e efeitos.

Função	Linhas	Papel
limitarCarroNaPista()	6455-6487	Mantém posição lateral e velocidade em intervalos válidos.
tocarSomColisao()	6488-6574	Aplica cooldown, som e mensagem de contato.
resolverColisaoEntreCarros()	6575-6963	Detecta sobreposição e resolve contato lateral ou longitudinal.
verificarColisoes()	6964-7061	Compara todos os pares e repete a resolução até três passadas.

Detecção

A diferença lateral dx e a diferença longitudinal dz são comparadas com `LARGURA_COLISAO_CARRO` e `COMPRIMENTO_COLISAO_CARRO`. Se qualquer uma estiver fora do limite, não há colisão. Se ambas estiverem dentro, existe sobreposição.

Contato lateral

Quando a geometria indica contato predominantemente lateral, os carros são afastados em direções opostas, perdem velocidade e, se forem rivais, recebem novos alvos de faixa para continuar a separação.

Contato traseiro

O carro de trás é colocado imediatamente atrás do carro da frente, recebe redução significativa de velocidade e uma parcela do movimento pode ser transferida ao veículo da frente. Um pequeno desvio lateral também ajuda a desfazer o engavetamento.

18. Voltas e melhor tempo

`verificarVoltaJogador()`, linhas 7062-7173, divide a distância percorrida pelo comprimento da pista e aplica `Math.floor` para descobrir quantos comprimentos completos já foram percorridos. Quando esse número aumenta, uma volta terminou.

O tempo da volta é calculado por `performance.now()` menos `inicioVolta`. O menor valor é armazenado em `melhorVolta`. Ao atingir `comprimentoPista * totalVoltas`, a função chama `finalizarJogador()`.

19. IA competitiva dos adversários

atualizarIA(), linhas 7174-8106, é o maior bloco de tomada de decisão. Ele percorre todos os rivais e calcula velocidade, turbo, mudança de faixa, previsão de curva, recuperação e chegada.

19.1 Previsão de curvas

A IA consulta três segmentos futuros, aproximadamente 650, 1650 e 2900 unidades adiante. A intensidade das curvas recebe pesos de 50%, 32% e 18%. Esse valor cria uma penalidade de velocidade antes que o carro chegue ao trecho mais difícil.

Combinação de curvas futuras

```
const curvaPrevista =  
  Math.abs(seg1.curve) * .50 +  
  Math.abs(seg2.curve) * .32 +  
  Math.abs(seg3.curve) * .18;
```

19.2 Velocidade-alvo

A velocidade-alvo parte do máximo do carro e é multiplicada por velocidadeIA, personalidade do rival, habilidade individual e penalidade de curva. Assim, o mesmo traçado produz comportamentos diferentes entre dificuldades e entre rivais.

19.3 Recuperação competitiva

diferencaParaJogador compara a distância do rival com a do participante. Se o rival estiver muito atrás, bonusRecuperacao cresce em estágios e aumenta o alvo. Se estiver muito à frente, o alvo recebe uma redução pequena. Nenhuma dessas regras altera diretamente a posição do carro.

19.4 Turbo da IA

Cada rival tem turbo próprio. A decisão considera reta, perseguição, proximidade da disputa, carga, velocidade mínima e parâmetro turboIA. O turbo é desligado quando a curva fica forte ou a carga chega perto do fim.

19.5 Ultrapassagem e espaço lateral

Periodicamente a IA procura um bloqueador à frente. Se encontrar, cria dois candidatos de faixa, um à esquerda e outro à direita. Para cada candidato, mede a menor folga em relação aos veículos próximos e escolhe a alternativa mais livre.

19.6 Movimento lateral

O rival não salta diretamente para alvoX. Ele aproxima ai.x do alvo com velocidade lateral limitada. Isso cria uma mudança de faixa gradual e permite que a colisão continue funcionando durante a manobra.

20. Ordem, posição e cronômetro

`participantesOrdenados()`, linhas 8107-8212, cria uma lista com participante e rivais. Quem já terminou é classificado por tempo final; quem ainda corre é classificado pela distância. `posicaoJogador()`, linhas 8213-8235, localiza o participante nessa lista.

`tempoCorridaAtual()`, linhas 8236-8265, calcula o tempo total descontando `tempoPausadoAcumulado`. Isso mantém a pausa fora do resultado competitivo.

21. Finalização, pontuação e gravação

finalizarJogador(), linhas 8266-8544, executa apenas uma vez. Marca o participante como terminado, registra tempo, identifica posição, interrompe o som do motor, toca som de vitória quando necessário, calcula pontos, preenche a janela final e salva o resultado.

Fórmula de pontos

```
const cfg = CONFIG_DIFICULDADE[dificuldadeAtual];
const pontos = Math.max(10, Math.round(cfg.premio * (9 - pos) / 8));
```

O fator $(9 - \text{pos}) / 8$ vale 1 para primeiro lugar e diminui conforme a posição. O prêmio-base muda por dificuldade. Depois do cálculo, Banco.salvar() recebe todos os dados necessários para ranking e histórico.

Campo salvo	Origem
nome	jogador.nome
data	new Date().toISOString()
posicao	posicaoJogador()
tempo	jogador.tempoFim
melhorVolta	melhor tempo registrado
velocidadeMax	jogador.velMax
dificuldade	dificuldadeAtual
voltas	totalVoltas
pontos	fórmula de pontuação
cenario	cenarioAtual
carro	carroAtual
modo	modoAtual

22. Renderização do quadro

`renderizar()`, linhas 8545-9010, organiza o desenho completo de um quadro. Ele obtém as dimensões atuais, posiciona a câmera em relação ao participante, desenha o cenário, percorre segmentos visíveis da pista, desenha adversários, efeitos de velocidade e o carro do participante ou o capô, depois chama `minimapa` e HUD.

Adversários na perspectiva

`desenharAdversariosNoSegmento()`, linhas 9011-9207, filtra rivais realmente dentro da distância de renderização e do segmento atual. A posição x, y e largura são interpoladas entre os pontos projetados do segmento. Isso evita que carros apareçam por causa da repetição modular da pista quando, na distância contínua, estão em outra volta.

Linhas de velocidade

`desenharLinhasVelocidade()`, linhas 9208-9323, adiciona traços laterais cuja quantidade e extensão dependem da intensidade de velocidade, aumentando a sensação visual de movimento.

23. HUD e minimapa

atualizarHUD() - linhas 9324-9523

Calcula posição, volta, km/h e tempo; atualiza textos; transforma a barra de turbo e a barra de RPM; indica turbo ativo; mostra câmera; monta o placar ordenado; e exibe mensagens quando a posição muda.

Atualização sem mudar a largura estrutural do HUD

```
hudTurbo.style.transform =  
  `scaleX(${(jogador.turbo / 100).toFixed(4)})`;
```

desenharMiniMapa() - linhas 9524-9779

O minimapa utiliza o segundo Canvas. Ele desenha uma forma fechada baseada no progresso angular do circuito e depois converte a distância de cada piloto em um ponto nessa forma. O participante recebe ponto maior e cor destacada.

escapeHtml() - linhas 9780-9809

Antes de inserir nomes dentro de innerHTML, escapeHtml() substitui caracteres especiais. Isso evita que um nome digitado no campo seja interpretado como marcação HTML.

24. Loop principal com passo fixo

loop(), linhas 9810-9909, é chamado por requestAnimationFrame. O tempo real entre quadros é colocado em acumulador. Enquanto houver pelo menos PASSO_FIXO acumulado, a física avança em blocos de 1/60 de segundo.

Sequência de atualização da simulação

```
while (acumulador >= PASSO_FIXO) {  
  if (estado === 'corrida') {  
    atualizarJogador(PASSO_FIXO);  
    atualizarIA(PASSO_FIXO);  
    verificarColisoes();  
    verificarVoltaJogador();  
  }  
  acumulador -= PASSO_FIXO;  
}
```

Por que usar passo fixo

A renderização pode variar conforme desempenho do computador, mas a simulação ganha estabilidade quando aceleração, colisão e decisões são processadas em intervalos constantes. requestAnimationFrame continua responsável por sincronizar o desenho com o navegador.

25. Pausa, menu, modais e mensagens

Função	Linhas	O que faz
pausar()	9910-9954	Muda estado para pausado, registra início da pausa, abre modal e reduz som do motor.
continuar()	9955-10006	Acumula duração da pausa, volta ao estado corrida, fecha modal e devolve foco.
voltarMenu()	10007-10063	Interrompe contagem, fecha janelas, troca telas e limpa estado visual de turbo.
mostrarRanking()	10064-10237	Lê resultados, agrega por nome e monta a tabela de classificação.
abrirModal() / fecharModal()	10238-10312	Controlam classes CSS e aria-hidden.
mostrarMensagem()	10359-10397	Exibe mensagens temporárias sobre a corrida.
ajustarCanvas()	10398-10528	Atualiza dimensões do Canvas apenas quando necessário.
atualizarPreviewMenu()	a partir de 10529	Atualiza nome, estatísticas, cor do mini carro e descrição do cenário no menu.

26. Eventos de teclado, toque e botões

A partir da região das linhas 10592 em `game.js`, `window.addEventListener` registra `keydown`, `keyup`, `blur` e `resize`. `Keydown` atualiza o objeto teclas, trata troca de câmera e pausa; `keyup` libera comandos; `blur` limpa estados para impedir teclas presas; `resize` chama `ajustarCanvas()`.

Controles de toque

Botões com data-controle são associados a `pointerdown`, `pointerup`, `pointercancel` e `pointerleave`. Essa escolha funciona com toque, caneta e mouse no mesmo modelo de eventos.

Botões do menu e modais

Os listeners ligam `btnJogar` a `iniciarCorrida()`, `btnRanking` a `mostrarRanking()`, botões de pausa às respectivas funções e selects de carro e cenário a `atualizarPreviewMenu()`. Assim, o HTML permanece declarativo e a lógica fica concentrada no JavaScript.

27. Como os sistemas se conectam

Origem	Destino	Exemplo de integração
Menu HTML	Variáveis do jogo	iniciarCorrida() lê modo, dificuldade, voltas, cenário e carro.
CARROS	Física	atualizarJogador() usa max, turboMax, aceleração, frenagem e controle.
CARROS	IA	atualizarIA() usa as estatísticas do carro de cada rival.
CENARIOS	Pista	construirPista() escolhe sequência de curvas e elevações.
CENARIOS	Renderização	desenharCenario() escolhe o fundo e a paleta participa do desenho da pista.
CONFIG_DIFICULDADE	IA	Define ritmo, erro, agressividade, turbo e recuperação.
Física	HUD	Velocidade, posição, turbo, volta e tempo são lidos a cada quadro.
Finalização	IndexedDB	Resultado completo é gravado por Banco.salvar().
IndexedDB	Ranking	mostrarRanking() agrega corridas e ordena pilotos.

A principal vantagem dessa organização é evitar sistemas paralelos. O mesmo conjunto de dados alimenta física, IA, interface e resultado, reduzindo divergências.

28. Onde modificar cada recurso

Objetivo	Onde alterar	Cuidados
Criar novo carro	Objeto CARROS no início de game.js; adicionar option em index.html.	Definir max, turboMax, aceleração, frenagem, controle, cor e tipo visual.
Alterar dificuldade	CONFIG_DIFICULDADE.	Testar ritmo, turbo e recuperação em várias pistas.
Criar novo cenário	CENARIOS, option do HTML, construirPista() e nova função de desenho.	Manter chave consistente em todos os pontos.
Mudar traçado	construirPista(), linhas 2201-2579.	Equilibrar retas, curvas e elevação; testar IA e câmera.
Mudar física	atualizarJogador(), linhas 6029-6454.	Alterações podem afetar tempo, IA automática e colisões.
Mudar IA rival	atualizarIA(), linhas 7174-8106.	Testar fácil, médio e avançado; evitar aceleração impossível.
Mudar colisões	constantes de colisão e funções 6455-7061.	Revalidar grade, ultrapassagens e engavetamentos.
Mudar pontuação	finalizarJogador(), linhas 8332-8353.	Ranking continuará somando os pontos gravados.
Mudar HUD	index.html + style.css + atualizarHUD().	Preservar IDs utilizados pelo JavaScript.
Mudar controles	modal Como Jogar + listeners + atualizarJogador().	Manter documentação e lógica sincronizadas.
Mudar câmeras	vetor CAMERAS e renderizar().	Verificar projeção, tamanho do carro e visibilidade da pista.
Mudar ranking	Banco + mostrarRanking().	Lembrar que registros antigos podem não conter novos campos.

29. Como adicionar um novo carro - roteiro técnico

9. Adicione uma nova entrada no objeto CARROS com uma chave única.
10. Defina nome, cor, tipo, max, turboMax, aceleracao, frenagem, controle e estatísticas.
11. Inclua uma nova option no select id="carro" em index.html usando a mesma chave no value.
12. Se quiser um desenho realmente diferente, amplie as variações tratadas por desenharCarro().
13. Abra o menu e confirme que atualizarPreviewMenu() mostra nome e estatísticas.
14. Teste aceleração, frenagem, direção, turbo, drift e colisões.
15. Teste o carro no modo manual e no Piloto automático - IA.

Estrutura compatível com o catálogo CARROS

```
novo: {  
  nome: 'Novo Carro',  
  cor: '#ffffff',  
  tipo: 'sport',  
  max: 315,  
  turboMax: 360,  
  aceleracao: 1800,  
  frenagem: 3300,  
  controle: 1.05,  
  estatisticas: 'Velocidade 315 • Aceleração 90 • Controle 86'  
}
```

30. Como adicionar um novo cenário - roteiro técnico

16. Crie uma entrada em CENARIOS com nome, descrição, tipo e paleta.
17. Adicione a opção correspondente no select id="cenario" do HTML.
18. Inclua um ramo em construirPista() com sua sequência de adicionarTrecho().
19. Crie uma função desenharCenarioNovo(largura, altura) para o fundo.
20. Atualize desenharCenario() para chamar a nova função.
21. Ajuste o minimapa caso queira um fator visual específico.
22. Teste curvas, elevações, distância de desenho, adversários, câmera e desempenho.

O segredo para manter consistência é usar a mesma chave do cenário em HTML, CENARIOS, construirPista() e desenharCenario().

31. Testes essenciais depois de alterar o código

Área	Teste recomendado
Inicialização	Abrir index.html e iniciar corrida com todas as combinações principais de menu.
Controles	Testar W/S/A/D, setas, Espaço, Shift, C, P e Esc.
Toque	Testar pointerdown e liberação dos botões em tela sensível ao toque ou emulação.
Carros	Confirmar diferenças de velocidade, aceleração e controle.
Cenários	Percorrer uma corrida completa em cada pista e observar curvas/elevações.
IA	Testar três dificuldades e verificar se rivais aceleram, freiam, usam turbo e ultrapassam.
Colisões	Testar contato lateral, traseiro, múltiplos carros e largada.
Voltas	Verificar 2, 3 e 5 voltas, melhor volta e fim exato.
Pausa	Pausar por alguns segundos e confirmar que esse tempo não entra no cronômetro competitivo.
Ranking	Finalizar corridas, abrir ranking, somar pontos e limpar dados.
Câmeras	Alternar as três em movimento e durante turbo.
Responsividade	Redimensionar janela e testar interface em largura reduzida.
Desempenho	Observar se Canvas não é redimensionado a cada quadro e se o HUD permanece estável.

32. Mapa final das funções principais

Função	Linhas	Responsabilidade
construirPista	2201-2579	Traçado conforme cenário
projetar	2613-2690	Perspectiva mundo -> tela
desenharSegmento	2760-2978	Pista e faixas
desenharCarro	4682-5131	Veículos
criarAdversarios	5207-5380	Grade e rivais
iniciarCorrida	5381-5596	Preparação geral
iniciarContagem	5597-5731	Contagem regressiva
obterControleAutomatico	5732-6028	IA do carro selecionado
atualizarJogador	6029-6454	Física e comandos
resolverColisaoEntreCarros	6575-6963	Contato físico
verificarColisoes	6964-7061	Todos os pares
verificarVoltaJogador	7062-7173	Voltas e melhor volta
atualizarIA	7174-8106	Rivais competitivos
participantesOrdenados	8107-8212	Classificação
finalizarJogador	8266-8544	Resultado, pontos e save
renderizar	8545-9010	Quadro visual
atualizarHUD	9324-9523	Painel de informações
desenharMiniMapa	9524-9779	Mapa da corrida
loop	9810-9909	Ciclo da simulação
mostrarRanking	10064-10237	Classificação persistida
ajustarCanvas	10398-10528	Dimensões e qualidade do Canvas

Encerramento técnico

O jogo combina uma pista segmentada em pseudo-3D, física por passo de tempo, IA com previsão de curvas e tomada de faixa, colisões entre todos os carros, renderização em Canvas, interface responsiva e persistência local. A separação em dados, funções e camadas permite modificar cada recurso sem reescrever toda a aplicação.